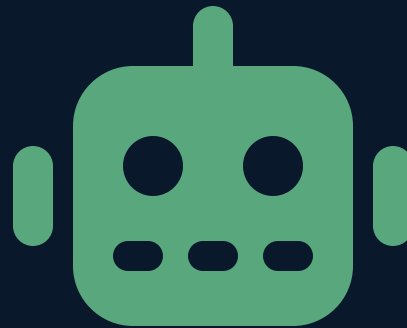


PORTADA

Técnicas de Programación

Unidad 6 — Sistema de Generación Automática de Reportes con IA

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

Un medio publicó decenas de artículos financieros escritos por IA — más de la mitad tuvo que corregirse por errores de cálculo o de hechos.



CASO REAL

CNET y los artículos financieros generados por IA (enero de 2023)

Dato	Detalle
Qué pasó	El medio tecnológico CNET publicó decenas de artículos explicativos sobre finanzas personales generados por un sistema de IA, marcados con una nota de autoría genérica.
Cómo se descubrió	El medio <i>Futurism</i> investigó y encontró errores factuales y de cálculo (ej. fórmulas de interés compuesto mal aplicadas) en una parte significativa de las notas publicadas.
Respuesta de CNET	La editorial confirmó el uso de IA, revisó su banco de artículos y publicó correcciones en más de la mitad de las piezas generadas por el sistema.

CASO REAL

La lección de fondo

El sistema de IA de CNET no falló porque la tecnología fuera inútil — falló porque los reportes se **publicaron sin una revisión humana rigurosa** antes de salir al público, en un dominio (finanzas personales) donde un error numérico tiene consecuencias reales para quien lee.

La conclusión para el sistema que van a diseñar hoy: un generador de reportes con IA es una herramienta que acelera la redacción, no un reemplazo del criterio de quien construye y valida el reporte final.

TRANSICIÓN

De licenciar y empaquetar, a generar reportes con IA

En la sesión pasada aprendieron a empaquetar y desplegar una librería propia con responsabilidad. Hoy usan ese mismo criterio de responsabilidad en otro contexto: construir un sistema que combina los datos que ya saben procesar (Unidad 4: CSV, Commons Math, JFreeChart) con un modelo de lenguaje que redacta la narrativa del reporte.

CONCEPTO

Arquitectura de un sistema de reportes con IA

Un sistema de generación automática de reportes con IA sigue un flujo de cuatro pasos, cada uno apoyado en algo que ya construyeron este semestre:

1. **Recolectar datos:** leer el CSV/JSON de origen (Unidad 4).
2. **Calcular:** obtener estadísticas descriptivas con Apache Commons Math (Unidad 4/6).
3. **Redactar:** enviar esos datos calculados a un modelo de lenguaje (LLM) para que genere el texto narrativo del reporte.
4. **Publicar:** combinar texto + gráficos (JFreeChart) en un documento final con PDFBox.

CONCEPTO

¿Qué es un LLM y cómo se consume desde Java?

Un **LLM** (*Large Language Model*, modelo de lenguaje de gran escala) es un modelo de IA entrenado para generar texto a partir de una entrada de texto (*prompt*). Los proveedores de LLM exponen su modelo como un **servicio web (API)**: la aplicación Java no ejecuta el modelo localmente, sino que le envía una petición HTTP con el prompt y recibe el texto generado como respuesta.

Es el mismo patrón cliente-cualquier-API-REST que ya usarán en la Unidad 7 con JavaScript y SpringBoot — aquí se consume desde Java puro.

CONCEPTO

Llamar a la API desde Java: HttpClient

Java trae un cliente HTTP incorporado desde la versión 11 (`java.net.http.HttpClient`), suficiente para enviar el prompt y recibir la respuesta sin agregar una librería externa:

```
HttpClient cliente = HttpClient.newHttpClient();

String cuerpoJson = ""
    {"prompt": "Redacta un párrafo resumiendo que el promedio
                de ventas de marzo fue 4.2M con desviación 0.8M"}
    "";

HttpRequest petition = HttpRequest.newBuilder()
    .uri(URI.create("https://api.proveedor-llm.com/v1/generar"))
    .header("Content-Type", "application/json")
    .header("Authorization", "Bearer " + apiKey)
    .POST(HttpRequest.BodyPublishers.ofString(cuerpoJson))
    .build();

HttpResponse<String> respuesta = cliente.send(
    petition, HttpResponse.BodyHandlers.ofString());
```


CONCEPTO

La clave de API no se escribe en el código

El parámetro `apiKey` del ejemplo anterior es una credencial secreta: quien la tenga puede consumir el servicio a nombre de otra persona, generando costos. **Nunca se escribe directamente en el código fuente** —y mucho menos se sube a un repositorio público—, sino que se lee de una **variable de entorno**:

```
String apiKey = System.getenv("LLM_API_KEY");
```

La misma regla aplica a cualquier credencial: contraseñas de base de datos, tokens de GitHub, claves de cualquier API — nunca hardcodeadas.

CONCEPTO

Diseñar el prompt: de datos crudos a instrucción clara

Un **prompt** bien diseñado no le pide al modelo "escribe algo sobre estos datos" — le da **contexto, datos concretos y el formato de salida esperado**, igual que un método bien diseñado recibe parámetros explícitos en vez de adivinar qué necesita:

Prompt débil	Prompt bien diseñado
"Analiza estas ventas."	"Con estos datos (promedio: 4.2M, desviación: 0.8M, mes: marzo), redacta un párrafo de máximo 60 palabras para un reporte ejecutivo, en tono formal, sin inventar cifras adicionales."

CONCEPTO

El riesgo central: alucinaciones

Un LLM genera texto **estadísticamente plausible**, no texto verificado contra una fuente de verdad — puede producir una cifra, un nombre o una conclusión que **suena correcta pero es falsa**. A eso se le llama **alucinación**. El caso de CNET (diapositiva 3) es exactamente esto: texto que parecía coherente pero contenía errores de cálculo reales.

Mitigación práctica en el sistema de hoy: el LLM solo redacta el texto *a partir de* los números que ya calculó Java con Apache Commons Math — nunca se le pide al LLM que calcule él mismo el promedio o la desviación estándar.

CONCEPTO

Human-in-the-loop: dónde entra la revisión humana

Human-in-the-loop (humano en el circuito) es el patrón de diseño donde el sistema automatizado no publica directamente su resultado, sino que lo deja en un estado de **borrador pendiente de aprobación** antes de convertirse en el PDF final:

```
ReporteBorrador borrador = generarConIA(datosCalculados);  
if (borrador.necesitaRevision()) {  
    guardarComoPendiente(borrador);    // no se publica todavía  
} else {  
    publicarComoPdf(borrador);  
}
```

Para un reporte académico o de bajo riesgo puede bastar con validación automática; para uno financiero o legal, la revisión humana antes de publicar no es opcional.

INTERACCIÓN

Encuentren el riesgo

En parejas: van a construir un sistema que genera automáticamente el boletín de notas de cada estudiante, incluyendo un párrafo de la IA con "recomendaciones para mejorar". Identifiquen un riesgo concreto de alucinación en ese párrafo, y propongan una mitigación (basada en lo visto hoy).

5 minutos · piensen en qué pasa si la IA "inventa" una nota o un dato del estudiante que no está en el CSV de origen

TRANSICIÓN

El sistema depende de librerías — elegirlas bien también es parte del diseño

Este sistema de reportes combina, como mínimo, cuatro librerías externas: una para HTTP (o el cliente de una API de IA), Commons CSV/Math, JFreeChart y PDFBox. La sesión pasada vieron cómo **licenciar y empaquetar** lo propio — ahora cierra el círculo: ¿con qué criterio se elige una librería **ajena** antes de agregarla al proyecto?

CONCEPTO

Checklist para evaluar una librería de terceros

1. **Licencia compatible:** ¿su licencia (MIT, Apache 2.0, GPL...) permite el uso que se le va a dar, incluyendo redistribución si aplica? (sesión anterior)
2. **Mantenimiento activo:** ¿tiene commits y releases recientes, o lleva años sin actualizarse?
3. **Salida accesible:** si la librería genera un gráfico o reporte visual, ¿permite no depender solo del color, exportar texto alternativo, o controlar tamaños de fuente? (retomando el software inclusivo de la Unidad 4)
4. **Tamaño de la comunidad:** ¿hay documentación, ejemplos y respuestas a problemas comunes, o es un proyecto casi sin usuarios?



CONCEPTO

Librerías inclusivas de análisis y visualización, revisadas con este checklist

Retomando JFreeChart y Apache Commons Math de la Unidad 4 bajo el checklist de la diapositiva anterior: ambas son **Apache 2.0** (permisiva, sin conflicto de licencia con un reporte comercial), tienen mantenimiento activo, y JFreeChart permite configurar colores, patrones de relleno y tamaños de fuente para cumplir los criterios de accesibilidad vistos en la Unidad 4 — no depender solo del color, buen contraste, texto legible.

Elegir una librería nunca es solo "¿hace lo que necesito?" — es también licencia, mantenimiento y qué tan accesible puede quedar lo que produce.



SÍNTESIS — CIERRE DE LA UNIDAD 6

Lo que hay que llevarse de la sesión y de la unidad

1. Un sistema de reportes con IA sigue el flujo recolectar → calcular → redactar (LLM) → publicar, consumiendo el LLM como una API HTTP con `HttpClient` y la clave leída de una variable de entorno, nunca hardcodeada.
2. Un prompt bien diseñado da contexto, datos concretos y formato esperado — y el LLM redacta a partir de números ya calculados en Java, nunca los calcula él mismo.
3. Las alucinaciones son el riesgo central (caso CNET): el patrón *human-in-the-loop* deja el reporte como borrador pendiente de revisión antes de publicarlo.
4. Elegir una librería de terceros combina cuatro criterios: licencia compatible, mantenimiento activo, salida accesible y comunidad — cerrando el vínculo entre licenciamiento (sesión anterior) y software inclusivo (Unidad 4).



CIERRE

Antes de la próxima sesión

- Repasa las dos sesiones de la Unidad 6 (licenciamiento/empaquetado/despliegue y reportes con IA) — son la base directa del **Caso de Estudio 2**.
- Piensa en un dato o reporte de un proyecto propio del curso que podría beneficiarse de este flujo, e identifica qué librerías necesitaría y bajo qué licencia están.

Con esta sesión cierra la Unidad 6 de Técnicas de Programación.

